

Transport Framing, Latency & Throughput, Worked Out by Hand

Why stdio is near-free and HTTP pays handshakes — and how Little’s Law turns a slow per-call latency into high throughput, derived on real numbers.

What this document does. MCP rides on two transports: stdio (a local pipe) and Streamable HTTP (TCP + TLS over the network). They answer the same three questions — framing, direction, reliability — at very different costs. This document counts the latency of each, amortises connection setup over a session, and then derives the single most useful systems law for a pipelined client: $\text{throughput} = \text{concurrency} / \text{latency}$ (Little’s Law).

Concepts covered: newline framing · round-trip latency · TLS/TCP handshake cost · setup amortisation · Little’s Law $L = \lambda W$ · serial vs pipelined throughput · bandwidth.

Internal learning material. v1.0

1 The setup: framing first, then the clock

stdio frames messages as **newline-delimited JSON**: one UTF-8 JSON object per line, separated by `\n`. To find a message boundary you scan for the next `0x0A` — an $O(b)$ pass over b bytes. A modern CPU scans memory at ~ 10 GB/s, so framing a 200-byte message costs

$$\frac{200 \text{ B}}{10 \times 10^9 \text{ B/s}} = 2 \times 10^{-8} \text{ s} = 20 \text{ ns}.$$

Framing is never the bottleneck. The clock that matters is the *round trip*.

Intuition

The protocol asks the transport for only three things: where one message ends (framing), the ability for both sides to send at will (direction), and in-order intact delivery (reliability). Everything expensive — TLS, retransmission, the network itself — lives below that line. So “which transport” is really “how far away is the server, and what must I pay to reach it.”

2 Step 1 — Per-call latency, stdio vs HTTP

A request–response round trip costs, for each transport:

$$W_{\text{stdio}} = \underbrace{t_{\text{write}} + t_{\text{sched}} + t_{\text{read}}}_{\text{kernel pipe, same host}} \approx 0.05 \text{ ms}, \quad W_{\text{http}} = \underbrace{\text{RTT}}_{\text{network}} + t_{\text{server}} \approx 25 \text{ ms} + 25 \text{ ms} = 50 \text{ ms}.$$

stdio copies bytes between a parent and child process through a kernel pipe — microseconds. HTTP must cross the network (one round-trip time, here $\text{RTT} = 25$ ms) and wait on the server. The local pipe is roughly $1000\times$ faster *per call*.

3 Step 2 — Connection setup, and amortising it

stdio pays a one-time process spawn (~ 30 ms) and then nothing. HTTP with TLS 1.3 pays, before the *first* request:

$$\text{setup} = \underbrace{1 \text{ RTT}}_{\text{TCP SYN}} + \underbrace{1 \text{ RTT}}_{\text{TLS 1.3}} = 2 \times 25 \text{ ms} = 50 \text{ ms}.$$

On a keep-alive connection that setup is paid *once* and amortised over all n requests in the session. The effective per-request overhead is

$$\text{overhead}(n) = \frac{50 \text{ ms}}{n}.$$

Mini-example — this operation in isolation

Send $n = 1$ request: it carries the full 50 ms setup, so it costs $50 + 50 = 100$ ms. Send $n = 100$ on the same connection: setup amortises to $50/100 = 0.5$ ms each, and the per-call cost falls back toward the 50 ms round trip. *Reusing the connection* is what makes HTTP viable — a new TLS handshake per call would double every request.

Watch out

“HTTPS is slow” is a statement about *setup*, not steady state. If your client opens a fresh connection per tool call you pay 2 RTT every time; with keep-alive you pay it once. The fix is connection pooling, not abandoning TLS.

4 Step 3 — Throughput: Little’s Law

Per-call latency W looks discouraging ($50 \text{ ms} \Rightarrow$ only 20 calls/s if you wait for each). But JSON-RPC lets many requests be in flight at once (Module 1.0). **Little’s Law** relates the three quantities of any stable queueing system:

$$\boxed{L = \lambda W} \quad \Rightarrow \quad \lambda = \frac{L}{W},$$

where L is the average number of in-flight requests (concurrency), W is the average latency per request, and λ is the throughput (requests/second).

Plugging in $W = 50 \text{ ms} = 0.05 \text{ s}$:

$$\begin{aligned} L = 1 \text{ (serial)} : \quad \lambda &= 1/0.05 = 20 \text{ req/s}, \\ L = 10 \text{ (pipelined)} : \quad \lambda &= 10/0.05 = 200 \text{ req/s}, \\ L = 50 : \quad \lambda &= 50/0.05 = 1000 \text{ req/s}. \end{aligned}$$

The latency of any *single* call did not change — you raised throughput purely by keeping more requests in flight.

in-flight L	latency W	throughput $\lambda = L/W$
1	50 ms	20 req/s
5	50 ms	100 req/s
10	50 ms	200 req/s
25	50 ms	500 req/s
50	50 ms	1000 req/s

Intuition

Latency and throughput are different axes. Latency is how long *one* request takes; throughput is how many *finish per second*. A pipeline that holds L requests “in the air” completes one roughly every W/L seconds even though each still takes the full W . This is exactly why the `id` field (Module 1.0) matters: without it, L is forced to 1 and Little’s Law caps you at $1/W$.

Watch out

Little’s Law gives the *ceiling*, not a guarantee. Real throughput is capped by whichever runs out first: client concurrency L , server worker pool, or network bandwidth. Raising L past the server’s parallelism just grows the queue and inflates W — the law still holds, but now W is rising with L and λ flattens.

5 Step 4 — Bandwidth: the other ceiling

Throughput in requests/second meets a second wall in bytes/second. At $\lambda = 1000$ req/s with 200-byte messages each way:

$$\text{bandwidth} = 1000 \text{ req/s} \times (200 + 200) \text{ B} = 4.0 \times 10^5 \text{ B/s} \approx 0.4 \text{ MB/s}.$$

Tiny for control traffic — so for typical MCP messages the binding constraint is RTT and concurrency, not bandwidth. Bandwidth only dominates when results are large (a multi-megabyte *resources/read*), at which point you stream.

6 The argument, at a glance

#	Idea	Formula	Value
0	Newline framing cost	$b/\text{scanrate}$	20 ns
1	stdio round trip	kernel pipe	~ 0.05 ms
1	HTTP round trip	$\text{RTT} + t_{\text{srv}}$	~ 50 ms
2	TLS + TCP setup	2 RTT	50 ms
2	Amortised overhead	setup/n	0.5 ms @ $n=100$
3	Throughput (Little’s Law)	$\lambda = L/W$	200 @ $L=10$
4	Control bandwidth	$\lambda \cdot \text{size}$	0.4 MB/s

7 Equation cheat-sheet

Framing scan cost	$b/\text{scanrate}$ (bytes / B-per-s)
HTTP round trip	$W = \text{RTT} + t_{\text{server}}$
TLS 1.3 + TCP setup	2RTT
Amortised setup per request	setup/n
Little's Law	$L = \lambda W$
Throughput	$\lambda = L/W$
Serial-throughput cap	$\lambda_{\text{max}} = 1/W$ (when $L = 1$)
Bandwidth	$\lambda \times (\text{req} + \text{resp size})$

A Reference implementation

```

RTT = 0.025                                # 25 ms one-way round trip
T_SERVER = 0.025

def http_latency():                          # steady-state per call
    return RTT + T_SERVER                    # 0.050 s

def setup_overhead(n):                       # amortised TLS+TCP
    return (2 * RTT) / n

def throughput(L, W):                       # Little's Law lambda = L/W
    return L / W

print(throughput(1, http_latency()))        # 20.0 req/s
print(throughput(10, http_latency()))      # 200.0 req/s
print(setup_overhead(100))                  # 0.0005 s = 0.5 ms

```

— end of document —